POST-DELIVERY ANALYSIS & DEVELOPER GUIDE

Logistics Vertical Restructure

One logistics domain became a tiered vertical — a shared base plus product domains that can be sold, activated and released independently. What changed, what it cost, and what you must do with in-flight work.

Delivered: 11 modules re-tiered into **8 base + 3 product**, two kernel defects fixed, two shared shapes extracted. **Zero data migration** — existing tenants auto-migrate on the normal upgrade path.

If you have a branch open: read §5. For most branches the answer is "almost nothing" — model keys did not change, so 284 references across 72 files needed no edit at all.

AUDIENCE
Engineering team

BRANCH
main-multi_domain_restructuring-dharmang

DATE
31 July 2026

SECTION 01

01 What changed

The single `logistics` domain is now a **tiered vertical**: a base domain holding what every logistics product needs, and product domains for each business line. Products depend on the base; they can never depend on each other.

BASE TIER **logistics_base** *What both products need. Depends only on the platform.*	masters · equipment_operations · pricing · sales_crm · booking · shipments · operations_execution · profitability
PRODUCT TIER **logistics_freight_forwarding** *Freight forwarding only.*	consolidation · documentation · tracking
PRODUCT TIER **logistics_nvocc** *Not built yet — the next product to land.*	(future)

11	0	0	2	47
MODULES RE-TIERED	MODEL KEYS RENAMED	DATA MIGRATIONS	KERNEL DEFECTS FIXED	NEW TESTS

Activation is now the product lever

```
ACTIVE_DOMAINS = ["logistics_base", "logistics_freight_forwarding"]   # base FIRST
BASE_DOMAINS   = ["logistics_base"]
```

A customer buying one product activates the base plus that product. Adding NVOCC later is an additive entry in the same list — no change to the freight product.

> **Why base-first matters**
>
> Domain load order **is** the order of this list, and the loader rejects a dependency that is not yet loaded. Putting a product domain before the base fails the boot with a message naming the missing key — the ordering is self-enforcing.

SECTION 02

02 The analysis behind it

Three findings shaped the work. Each came from measuring the codebase rather than reading the plan — and each changed what got built.

1 · The dependency graph decided the split, not preference

Every module's declared dependencies were resolved transitively. **Putting a module in the base forces its entire dependency closure into the base with it.** Only four modules are reverse-leaves — nothing depends on them — so only those could stay behind:

Module	Depended on by	Consequence
masters	all 10 others	Must be base. Moves first — no dependencies of its own.
shipments	consolidation, documentation, operations_execution, profitability, tracking	Not optional — dragging booking, sales_crm, pricing, equipment_operations with it.
documentation · operations_execution · profitability · tracking	`nothing`	The only free choices.

Of those four, `documentation` and `tracking` depend on `consolidation`, whose co-load model is freight-specific — so those three travel together and form the product tier.

2 · Two kernel defects, both found by probing rather than reading

Defect	Detail
Peer coupling was accepted	Dependency validation only checks that a dependency is already *loaded* — it does not care which domain it came from. A product-to-product dependency **boots fine**, silently ending the one-directional graph. Fixed by a boot guard (§7).
Keyed abstracts crashed the boot	`@api.model(key, abstract=True)` is a documented parameter, but the decorator sets a key and returns early without registering, while the loader gates only on "has a key" and hands the class to `register_model`, which rejects abstracts. The flag had **zero usages**, so nothing had ever exercised it.

3 · Model keys and app keys are independent — and that made it affordable

The model key is declared in the decorator; the app key derives from folder position. Moving a module changes the app key and **nothing** about model identity. Keeping the keys preserved:

Untouched	Why
284 references across 72 files	Every `logistics.*` model key, domain string, command name and hook key still resolves
32 seed files	Data CSV *filenames* encode the model key, not the app key
Every XML id and view `<extend ref>`	Scoped by the **short app name**, which a domain move does not change
Alembic history	Revision ids are content identity, not path identity

SECTION 03

03 Production migration

Audited **before** a single module moved. An existing tenant upgrades on the normal path with **no data migration** — every surface that persists app identity either does not change or self-heals.

Surface	Behaviour on upgrade
Tables / schema	Unchanged. Model keys unchanged means table names unchanged.
Alembic history	Revision ids are content identity; version locations are rebuilt from the registry at runtime, and nothing reads the app-key header. Moving a migration file between app folders is transparent.
`ir.model.app_key`	Stores the **short** app name, not the full key. The row is upserted by an external id built from `(short_app_name, model_key)` — stable across a domain move — so it is **updated in place** by the registry sync that runs inside `ede migrate upgrade`.
`ir.data.reference` (XML ids)	Its module column is the short app name → unchanged → menus, actions, views, RBAC rows and demo refs all keep resolving.

⛔ The one thing that would break it — renaming a module

A rename changes the short app name, which orphans every `<old_module>.*` XML id. Orphan cleanup does not merely tidy those references:

```
no inbound FKs  ->  delete the record
```

So seeded masters, menus, actions and RBAC permission rows with no inbound foreign keys would be **deleted in production**. That is data loss, not untidiness.

For this reason the proposed `equipment_operations` → `equipment_control` rename was **dropped**. The module kept its name.

⚠ The standing rule this establishes

Moving a module between domains is free. Renaming a module is a data migration. Do not rename a module without one — and note the migration generator cannot produce a data migration, so it needs a deliberate, approved plan.

How the "no schema change" claim was proven

Not by argument. Each relocation was verified by running the real generator against the configured PostgreSQL tenant:

```
ede migrate generate --app logistics_base.booking   --config ede.conf  ->  0 app(s),  5 files unchanged
ede migrate generate --app logistics_base.shipments --config ede.conf  ->  0 app(s),  4 files unchanged
ede migrate generate --app logistics_base.masters   --config ede.conf  ->  0 app(s), 13 files unchanged
```

No revision file was written for any app — the tool that would emit a migration emitted nothing.

SECTION 04

04 What shipped, stream by stream

WS	Change	Detail
1	Domain tier guard	Boot validator reading domain and dependency data off registered app records. Base may depend on the platform only; a product may add a base domain and its own siblings; peer products never couple. Collects every violation and raises once. **13 tests.**
2	Keyed abstract registration	Registry gained its own abstract map; the loader routes abstracts there instead of to concrete registration; lineage is **derived from the inheritance chain**, not declared; boot validates it. **16 tests.**
3	Base domain + masters	New base domain; 42 masters moved wholesale. All were audited against both businesses and every one is needed by both.
4	5 core modules	equipment_operations, pricing, sales_crm, booking, shipments — moved in dependency order.
5	2 engine modules	operations_execution, profitability — reverse-leaves, moved last and independently.
6	Product domain rename	`logistics` → `logistics_freight_forwarding`. Wide but shallow: no schema, no data, no model-key change.
7	Shared shapes	`mixin.charge.line` (10 fields + margin behaviour) and `mixin.party.line` (5 fields), extracted *out of* shipped models. **18 tests.**

Two scope corrections made against measured evidence

Planned	What the code said
An NVOCC container-control module	All eight of the container/equipment deliverables target **base-domain** models, and the freight product already ships equipment tracking. Container capability is not agency capability — it belongs in the base. The module was dropped.
`mixin.package.measure`	Its two candidate consumers share field *names* but **not one shared definition** — precision/scale, defaults, read-only and delete-behaviour all differ, and volume is even named differently. Extracting it would re-shape a live column. The volumetric rule is already shared as a function with a single consumer, so there was no behaviour to lift either. Not built.

The rule both corrections express

Extract a shared shape only when a real second consumer exists **and** the definitions actually match. A shared base that neither consumer fits is worse than two honest copies — and if the definitions differ, "sharing" them silently re-shapes somebody's production column.

SECTION 05

05 If you have work in flight

For most branches the answer is **almost nothing**. Model keys did not change, so anything referencing a model by key still works untouched.

Still valid — do not touch

This still works	Because
`@api.model("logistics.equipment")` and every `logistics.*` key	Keys were deliberately not renamed
`env.models["logistics.facility.master"]`	Same
`fields.Reference("logistics.unlocode.master")`	Same
Command names, hook keys `pre.logistics.*.create`	Same
XML ids, `<extend ref="masters.view_id">`	Scoped by short app name — unchanged
Data CSV filenames	Encode the model key

What does change — four surfaces

All keyed to the module's new home: `logistics_base` for the 8 shared modules, `logistics_freight_forwarding` for the 3 product ones.

```
BASE='masters\|equipment_operations\|pricing\|sales_crm\|booking\|shipments\|operations_execution\|profitability'
PROD='consolidation\|documentation\|tracking'

# 1. manifest depends        2. python imports        3. path literals in tests
grep -rl "\"logistics\.\($BASE\)\"" src/ --include=__manifest__.py \
  | xargs -r sed -i "s/\"logistics\.\($BASE\)\"/\"logistics_base.\1\"/g"
grep -rl "domains\.logistics\.\($BASE\)" src/ --include=*.py \
  | xargs -r sed -i "s/domains\.logistics\.\($BASE\)/domains.logistics_base.\1/g"
grep -rl "src/domains/logistics/\($BASE\)" src/ --include=*.py \
  | xargs -r sed -i "s#src/domains/logistics/\($BASE\)#src/domains/logistics_base/\1#g"
# ... repeat the three with $PROD -> logistics_freight_forwarding

# 4. verify — nothing may remain under the retired domain
grep -rn 'domains\.logistics\.\|src/domains/logistics/' src/ --include=*.py \
  | grep -v logistics_base | grep -v logistics_freight_forwarding
```

> ⚠ **The trap that caught us twice**
>
> **Segmented** path constructions are invisible to the slash-form replace above:
>
> ```
> Path(__file__).resolve().parents[3] / "domains" / "logistics" / "pricing" / ...
> ```
>
> Grep them separately — `grep -rn '"domains" */ *"logistics"' src/ --include=*.py` — or they will fail only at test time.

Then run `./run_tests.sh`. A boot failure naming a dependency that "is not loaded yet" means a manifest still points at the old app key.

SECTION 06

06 Rules for new code

Seven rules. The first three are enforced at boot; the rest are judgement, with the reasoning attached so you can apply them to cases this document does not cover.

#	Rule	Why
1	**Never make one product domain depend on another**	Rejected at boot. If two products need the same thing, it belongs in the base — that is the whole point of the tier.
2	**Never make the base depend on a product**	Rejected at boot. It inverts the tier and creates a cycle in everything but name.
3	**List the base domain first** in the active-domain list	Load order is list order; a dependency must already be loaded when declared.
4	**When unsure, put it in the base**	The mistakes are not symmetric. A base module only one product uses is harmless over-inclusion. A product module the *other* product later needs leaves you blocked — or tempted into rule 1.
5	**Move modules freely; never rename one casually**	A rename changes the short app name, orphaning XML ids that orphan cleanup then deletes. See §3.
6	**Keep `logistics.*` keys for base-tier models**	Existing keys are grandfathered. New product-specific models take the product prefix, so the prefix tells you the tier for everything written from here on. Do not mass-rename.
7	**Prove a relocation with an empty generated migration**	The only mechanical proof that a move changed layout and not schema. A non-empty diff means something was edited in transit.

Using a shared shape

Declare a keyed abstract in the base and subclass it — fields, constraints **and behaviour** inherit normally:

```
from domains.logistics_base.masters.models.abstracts import ChargeLineMixin

@api.model("logistics.booking.charge")
class BookingCharge(ChargeLineMixin, DomainModel):
    booking_id = fields.Reference("logistics.booking", on_delete="cascade", required=True)
```

You want to…	Do this	Not this
Add a field only one consumer needs	Declare it on the **concrete**	—
Add a field **every** consumer needs	Edit the mixin — you own the base	The extension SDK: it cannot target an abstract
Add a field to a concrete owned by another module	The extension SDK, with your own migration	Editing their model file

> **Two behaviours worth knowing**
> On a field-name collision between two mixins, **the first base in the class's bases list wins — silently**. And a foreign key can never target an abstract: if something must reference "either kind" of a per-product record, use a shared concrete parent, not a mixin.

SECTION 07

07 Guardrails & verification

The structure is not held together by documentation. Three checks fail the build if it drifts.

Guard	Rejects
Domain tier validator *at boot*	A base domain depending on a product domain; two product domains depending on each other. Names the app, the dependency and the rule broken, and reports every violation at once.
Abstract lineage validator *at boot*	A concrete inheriting a `mixin.*` key whose module never loaded — almost always a missing manifest dependency.
Boot assertion in the test suite	Any app reappearing under the retired `logistics` domain. Pins all 8 base apps and all 3 product apps by name.
Schema-drift guard *shared shapes*	Any change to a concrete's field set after the mixin extraction, and any attempt to normalise the deliberately-divergent charge definitions. A future tidy-up fails the build rather than the database.

5888	673	47	3
PYTEST PASSING	VITEST PASSING	TESTS ADDED	APPS GENERATOR-VERIFIED

How each stream was verified

- Full suite run after **every** stream — not once at the end.
- Every failure encountered was a **stale test literal** (a hardcoded path or app key encoding the old layout), never a behavioural regression. Those were fixed as assertions, since the restructure was the intent.
- The empty-migration gate was run with the real generator against PostgreSQL, not inferred.
- Base-tier-only activation is a test: the base boots with no product domain present.

> **⚠ One thing not proven**
>
> The migration analysis in §3 was verified by reading the registry sync, the migration runner and the external-id model — and by the generator emitting nothing. It was **not** proven by upgrading a real tenant seeded on the old structure. If you want that evidence before a production rollout, it is a scratch-tenant test worth adding.

SECTION 08

08 What is not done

The restructure is complete and production-safe. The product it was built for is not yet built.

Item	State	Note
NVOCC domain + agency module	NOT STARTED	The second product domain: principal master and its lifecycle, entitlement, agency governance, numbering, partner registry, readiness gate. Unblocked — everything it depends on has landed.
Container control	NOT STARTED	Now **base** work, not NVOCC work: ownership-type master, movement-event crosswalk, ISO check digit behind a configuration switch, status guards. Extends the shared equipment registry in place.
Principal-wise data scoping	GATED	The remaining hard piece. The platform has a mature scoping mechanism for exactly one dimension — it injects an ownership filter on every read and fails closed. A principal is a second, orthogonal dimension. Generalising that mechanism is an approval-gated kernel change.
Tax, finance mapping, regional packs	BLOCKED	All depend on the platform localization module, which has not started. Sequence it ahead or descope.

Reference material

Document	Purpose
`docs/logistics-restructure-guide.md`	Living change log + the rebase recipe. Update it when you move a module.
`docs/nvocc-domain-enablement.md`	The architecture decision and its reasoning.
Developer work streams (PDF)	Per-stream build cards, including the two NVOCC streams still open.

The shape of the win

Adding NVOCC now changes nothing about how freight forwarding works — it activates alongside, over the same masters, pricing, booking, shipments and container registry. And because peer coupling fails at boot rather than in review, the next product after that is the same additive exercise. That property, not the folder layout, is what this restructure bought.